

Live Demo Runbook

Three minutes: one real Agent run, then every attack class you built against — 32 invariants in 11 seconds — closing on a containment receipt the judges verify themselves. Every command below is real and was timed on this tree.

SECTION 00

Read this before anything else

BLOCKING – VERIFY THE MERGE LANDED

The strongest material in this project — the confused-deputy exploit, authority attenuation, and signed termination receipts — came in through **PR 38**. As of writing, `origin/main` contained **none** of it: `demo-escalation.mjs`, `demo-kill.mjs`, `demo-tunnel-bypass.mjs` and `verify-receipt.mjs` were all absent from that branch.

Before you present, confirm the branch you are demoing from has all four. This entire runbook depends on them:

```
ls scripts/demo-escalation.mjs scripts/demo-kill.mjs \  
  scripts/demo-tunnel-bypass.mjs scripts/verify-receipt.mjs  
npm run check      # typecheck → build → tests, in that order
```

Status on merged `main`: all four scripts are present. 264 tests across 24 suites pass.

OPEN BLOCKER — FIX BEFORE SUBMITTING

`npm run check` **exits 1 even though every test passes.** An unhandled `RunCancelledError` escapes `container-codex-runner-pause.test.ts` — a cancelled run's promise is never awaited or caught, so Vitest reports *"Errors 1 error"* and fails the run.

This matters because `npm run check` **passing is on the track's own acceptance checklist.** A reviewer working that list hits a red on an otherwise green repository. It is a leaked promise in a test, not a containment defect — but fix it before submission, and do not invite judges to run `check` on stage until you have.

GOOD NEWS WORTH SAYING ON STAGE

Every demo script now **asserts** its invariants through `scripts/demo-assert.mjs` and exits non-zero if containment breaks. It is no longer a thing you read — it is a thing that can fail. If the README still says the demos "print observations rather than asserting them", that line is stale and should be corrected before submission.

SECTION 01

Fifteen minutes before you present

The biggest risk in a three-minute demo is watching a build or an image pull in silence.

☐ Pre-build the server.

Every `demo:*` script builds first. Build once now and the live runs start instantly.

☐ Run each demo once, end to end.

Caches the container images and proves the invariants hold on today's tree.

☐ Start the platform and leave it running.

`npm run poc` → `localhost:3000`. Never start it on stage.

☐ Create the Agent in advance.

Already in the list, lifecycle state visible. Creating one live burns twenty seconds on starter-kit CRUD.

- ☐ **Terminal font 18pt or larger.**

Judges read a projector from four metres. This demo is mostly terminal output — it has to be legible.

- ☐ **Two windows, no alt-tabbing.**

Browser left, terminal right, both visible at once.

- ☐ **Rehearse with a timer.**

The usual failure is running long and being cut off before the receipt — which is your best moment.

Pre-warm block

```
npm install
npm run build --workspace apps/server    # live demos then skip the build
node scripts/demo-escalation.mjs         # no engine, no key — always works
node scripts/demo-kill.mjs               # needs a container engine
docker ps                                # engine responds instantly?
npm run poc                               # leave running → localhost:3000
```

SECTION 02

The three-minute run of show

You run **every** demo. All five together take **11–13 seconds** of compute, so runtime was never the constraint — narration is. Run the whole gauntlet at once, then walk back through output that is already on screen, at whatever pace the clock allows.

MEASURED ON THIS TREE, JUST NOW

escalation 10/10 · egress 7/7 · tunnel-bypass 2/2 · kill 7/7 ·
passport 6/6

32 invariants across 5 attack classes in 11 seconds, every one machine-checked. Say that number out loud — it is the single most compressed proof of work you have.

0:00

Cold open — do not introduce yourself

SAY

“This is a coding agent doing its job. It read a GitHub issue. The issue told it to run `curl`. No exploit, no CVE — the agent did exactly what it was told, by the wrong person.”

DO

Browser only. Name the project after this lands.

0:15

Name it and frame it

SAY

“Prompt injection is unsolved, so we stopped trying to make the model trustworthy. Agent Passport is border control for agents: it does not judge the agent’s character, it decides where it may go — and revokes that before it boards.”

DO

Show the Playground, Agent already created, lifecycle state visible.

0:27

One real Agent run — the normal case (required by the brief)

SAY

“A real task through the real Codex runtime in a real container. Normal work is never interrupted — the border only matters when something tries to cross it.”

DO

Playground → `Run pwd, then list the workspace with ls -la.` Talk while it runs.

DO

Trace drawer: `run.started` → `step.auto_approved` → `ALLOW-STANDARD-000` → `run.completed`.

1:00

★ The gauntlet — every attack class, one command

SAY

“Now assume the agent is hijacked. Here is every way we could think of for it to get out, run against real containers and a real proxy. Watch the clock — this is the whole thing.”

DO

Paste the gauntlet from Section 03 and hit enter. **Do not narrate while it runs** — 11 seconds of silence with output flying past is more impressive than talking over it.

DO

When it lands, read the tally aloud: **“thirty-two invariants, five attack classes, eleven seconds. Every one held, and a broken boundary exits non-zero.”**

1:25

★ Walk the two that matter — scroll back up

SAY

On **escalation**: “Everyone builds the network wall, so we asked what a hijacked agent does *instead*. The control plane is reachable from inside the container, because the model adapter lives there. Its cheapest attack is not breaking the wall — it is asking the platform for a wider door. That worked, until we closed it. Authority only flows downhill.”

DO

Point at: it grants itself egress → **denied**. It claims to be the human → **denied**. It delegates to a sub-agent, then clones the grant back to itself → **denied**. Operator revokes the parent → the child **cascades**.

DO

Then one sentence each for the fast three: *“the network border holds under default-deny, revocation and quarantine; an opaque CONNECT tunnel to the control plane is refused; and cross-user access is denied before grants are even consulted.”*

2:10	★ The receipt — containment you don't take our word for SAY “Last question a security reviewer always asks: how do I know you actually contained it? You don’t have to trust us. Termination writes an Ed25519-signed receipt, and you verify it offline with the public key. This program does not get a vote.” DO Scroll to the kill block: freeze → block authority → kill → verify, then the agent unable to reach its target because authority is gone. DO Then the closer: the script tampers with the receipt and re-verifies — and it is rejected. “A public key lets you check a receipt. It does not let you forge one.”
2:45	Close, then stop talking SAY “Agents get real credentials the day something can take them back — and prove it did. We assumed the agent was compromised. Run it and watch it try.” DO Silence. Let them ask first.

Cut order, if you are running long

Decide this **before** you are on stage, not during. Cut strictly in this order:

CUT	WHAT GOES	SAVES
1	The one-sentence walk of the fast three (egress, tunnel, passport) — the tally already proved they passed	~20s
2	The Trace-drawer walk in the Agent run — just show it completed	~12s

- 3 The escalation detail — keep the headline “it asked for a wider door and was refused”, drop the four sub-beats ~18s

NEVER CUT

The gauntlet running live, the 32-invariant tally, and the tampered receipt being rejected. Those three are the submission.

IF TWO PEOPLE PRESENT

Speaker A: 0:00–1:00 (story, UI, normal case). Speaker B: 1:00–3:00 (gauntlet, walk-back, receipt, close). Hand over on “*now assume the agent is hijacked*”.

SECTION 03

The gauntlet, and what it prints

One command, five demos, every invariant asserted. Paste this exactly — the `grep` strips request logs that would otherwise scroll your results off the screen, and `pipefail` makes a failure inside the pipe still register.

```
set -o pipefail
for d in escalation egress tunnel-bypass kill passport; do
  printf '\n==== %s ==== \n' "$d"
  node scripts/demo-$d.mjs 2>&1 | grep -v '"level":[0-9]*,"time"' \
    || echo '!! INVARIANT FAILED – stop and look'
done
```

Expected tally, in order. If any number differs, something regressed — investigate before you present.

DEMO

HOLDS

WHAT IT PROVES

- 1 **escalation** 10/10 Authority only flows downhill: an agent cannot widen its own grants, impersonate the human, clone a delegated grant back to itself, or trade

capability families. Revoking a parent cascades to the child.

2	egress	7/7	Default-deny at the border, grant admits, revocation bites on the next connection, repeated probing quarantines, a guessed proxy credential fails, and a direct no-proxy attempt finds no route.
3	tunnel-bypass	2/2	An opaque CONNECT tunnel to the control plane is refused, and no authority is created through it.
4	kill	7/7	Freeze, atomically block authority and revoke, kill, verify — then a signed receipt, and an edited receipt is rejected.
5	passport	6/6	Identity and grants: no grant denies, a human grant allows, cross-user access is denied before grants are consulted, and revocation is felt on the next read.

Total: **32 invariants** across **5 attack classes** in **~11 seconds**.

If a judge wants to verify the receipt by hand

```
node scripts/verify-receipt.mjs receipt.json \  
  --public-key .data/receipt-signing-public.pem
```

It checks the trusted key ID, the signature, the exact step set and order, unique revoked-grant IDs, and that step outcomes agree with the `contained` claim. A public key permits verification but not forgery.

SECTION 04

When it breaks on stage

Work down this ladder without apologising. Recovering calmly reads as competence.

RUNG 1 – THE CONTAINER ENGINE IS DOWN

`demo-escalation.mjs` and `demo-passport.mjs` need **no engine and no key**. Your best demo is also your most robust one. Say: *“engine's not cooperating — here is the authority boundary, which does not need it.”*

RUNG 2 – THE LIVE AGENT RUN HANGS ON THE MODEL

Do not wait. Say: *“the provider is slow, which is exactly why containment cannot depend on the model,”* and go straight to the escalation demo. It never touches a model API.

RUNG 3 – NOTHING RUNS

Narrate over a **pre-recorded capture** and say plainly that it is a recording. Judges forgive a recording; they do not forgive two minutes of silence.

RUNG 0 – RECORD THE FALLBACK TONIGHT

Capture clean runs of `demo-escalation` and `demo-kill` while everything works. Five minutes now, and the worst case stops being catastrophic.

SECTION 05

Questions judges will ask

Grounded in what the code does. Where an answer needs checking against your merged branch, it says so.

? Isn't this just a firewall?

Lead with the escalation demo, not the network. **A firewall would not have stopped the actual attack** — the agent never touched the network boundary; it asked the control plane for a wider grant, from inside.

What stops that is an authority model: humans originate authority, agents may only ever pass along less, and capability families are incomparable so network access can never be traded out of storage access.

? How did you find the confused-deputy bug?

Be direct: the control plane is reachable from inside an agent container **because the model adapter lives there**. Once you see that, the cheapest attack is obviously not the wall — it is the door.

`demo-escalation.mjs` reproduces both the original exploit and the fixed behaviour, so the fix is regression-tested rather than asserted.

? What does the receipt actually prove?

Claim it narrowly and you will be believed: **the signed receipt authenticates the control plane's recorded termination observations** — origin, integrity, internal consistency, and that the step outcomes agree with the containment claim.

It does **not** claim earlier effects were undone, and offline verification cannot independently recreate the runtime observation. Saying that unprompted is worth more than overclaiming.

? Does revoking kill a connection that is already open?

No — and that is precisely why termination exists as a separate path. Revocation stops the next connection. Termination freezes first, then atomically blocks authority and revokes, then kills, then verifies. If a live runtime cannot be frozen, it records that failure and **refuses to claim containment**.

? Your human identity is still a mock.

Agree immediately and draw the line precisely. **Human identity is a mock** — `x-principal-id` is unauthenticated. **Agent identity is not**: it is bound to the containment topology, the proxy strips caller-supplied attestation and stamps the identity itself, and the agent container never holds the server secret.

Production needs an operator IdP and RBAC. The enforcement path above it does not change.

? What about DNS or another covert channel?

Contain the claim: HTTP and HTTPS go through the proxy, the internal network removes the direct route, and non-HTTP TCP is refused because it cannot traverse an HTTP proxy. Opaque CONNECT tunnels to the control plane are refused outright — `demo-tunnel-bypass.mjs`.

If you have not measured DNS on your merged branch, say so. A caught overclaim discounts everything else you said.

? Why hostname and not full URL for HTTPS?

CONNECT exposes only the host, so per-path rules need TLS termination. **Anthropic's own sandbox-runtime documents the same limit** — state of the art, not a shortcut.

? How much of this was the starter kit?

Precision reads as confidence. The approval UI, canary tripwire, budget breakers and trace timeline **came with the kit**. This project added identity, scoped and revocable grants, the authority-attenuation model, the network gate enforced before the socket opens, verified termination, and the signed receipts.

? What would you do next?

Have one specific answer: **authenticate the operator** (the last mock in the identity story), then **persist quarantine strikes**, which currently reset on restart. Both are named as limitations in the repo — you were tracking them, not discovering them under questioning.

SECTION 06

The three lines everyone must know cold

If the laptop dies and you are talking to a judge in a corridor, these still land.

THE PREMISE

“Prompt injection is unsolved, so we stopped trying to make the model trustworthy and made the platform safe while the model is hostile.”

THE INSIGHT

“Everyone builds the network wall. A hijacked agent's cheapest attack is not breaking it — it is asking the platform for a wider door. We found that hole and closed it: authority only flows downhill.”

THE PROOF

“Termination writes a signed receipt you verify offline with a public key. You do not have to trust our demo.”

SECTION 07

Everything else you shipped — claim it somewhere

The three-minute demo deliberately features two capabilities. These are the rest, merged and working. Do not cram them into the demo — use them in Q&A, and make sure every one appears in the written submission, because several are scored there.

PR CAPABILITY

WHERE TO CLAIM IT

48 HITL approval before egress side effects. An ungranted request is held at the proxy before the upstream socket opens; approval is request-scoped, so the next request asks again.

Q&A: “*what if the operator wants to allow just one thing?*” Reserve demo: `vitest run src/egress-hitl.integration.test.ts`

Cancellation and disconnect propagate so stale approvals are denied cleanly.

49 Approval attribution from server-issued sessions. Caller-supplied names are rejected; evidence records initiator, Agent, action, resource, decision, result, and the session actor who resolved it.

Q&A: “*who approved this, and can they lie about it?*” Visible in `demo:identity`.

46 Provider keys never reach the container argv. Config is passed by environment name; the Runtime authenticates with a generated internal adapter token; credentials are redacted from messages, run output, events, adapter errors, and streamed responses.

Put this in the README. “No secrets in source, history, logs, traces, screenshots, or demo output” is an explicit acceptance item — this is the evidence for it.

43 Output token budgets enforced before generation, with preventive controls distinguished from observational ones in API metadata, UI and docs.

Q&A: “*what stops a runaway agent burning your budget?*” The preventive/observational distinction is the same intellectual honesty as your telemetry labelling — say so.

42 Fail-closed pause and resume. Approval publishing requires a *verified* runtime pause; container pause state is checked rather than trusting the CLI. Missing or failed controls stop the run deterministically.

Q&A: “*what if the pause silently fails?*” Pairs with termination: you verify rather than assume, in both paths.

52 Provider failures are diagnosable. A revoked Gemini key returns Google's `400 INVALID_ARGUMENT`; it is re-labelled `401` naming the variable to check. An empty turn caused by reasoning exhausting the output cap becomes `502` naming the cap, instead of silently reporting success.

Only if asked, or if the demo misbehaves — it explains the failure instantly. Also a good “*what did you learn*” answer.

51 Credential hygiene. Placeholder or reused adapter tokens are rejected; canary tokens under 12 characters are refused.

One line in the README's security section.

IF PR 50 LANDS — MEMORY PASSPORT

PR 50 (*“give a belief a provenance, so a poisoned one arrives powerless”*) was still open at the time of writing, and none of its files were on `main`. If you merge it before submitting, **re-check this runbook**: memory provenance is a genuinely different capability from network and authority containment, and it deserves either its own sentence in the close or its own Q&A entry.

Do **not** add it to the three-minute demo. Two attacks well told beats three rushed.

Agent Passport · Track 1, Agent Launchpad · Step labels verified against `demo-escalation.mjs`, `demo-kill.mjs`, `demo-egress.mjs` and `demo-assert.mjs`.